



南開大學

《MPI 编程实验》

矩阵 SVD 分解算子 MPI 编程实验报告

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

2026 年 6 月 5 日

目录

1 实验背景	3
1.1 SVD 分解与本实验选题	3
1.2 本实验采用的 SVD 实现方式	3
1.3 MPI 编程实验目标	3
2 MPI 算法设计与实现	4
2.1 整体设计思路	4
2.2 项目文件结构与实现划分	5
2.3 GKH 迭代中的活动块划分	5
2.4 主从式结构与任务池设计	6
2.5 改变迭代方法: changed iteration	7
2.6 局部 GKH 迭代与 Givens 旋转记录	8
2.7 通信数据优化: 从 U/B/V 子块到 local B + rotation logs	9
2.8 master_work 策略	9
2.9 MPI blocking 与 nonblocking 通信探索	10
2.10 MPI 与 SIMD、OpenMP 的结合	10
2.11 主要数据结构与通信协议	11
2.12 Profiling 指标设计	12
2.13 小结	12
3 实验环境与测试规范	12
3.1 服务器与编译方式	12
3.2 运行参数与测试模式	13
3.3 正确性检查指标	13
3.4 Profiling 设计	14
4 正确性验证与服务器性能结果	14
4.1 实验数据说明与分析口径	14
4.2 正确性验证结果	15
4.3 MPI 策略演化实验	16
4.4 sweep_cap 调优结果	16
4.5 不同问题规模下的性能	17
4.6 不同进程数下的性能	17
4.7 不同节点布局下的性能	18
4.8 MPI+OpenMP 混合并行	18
4.9 blocking 与 nonblocking 通信对比	19
4.10 编译优化与 SIMD 选项探索	19
4.11 最终推荐配置与性能总结	20
5 结论与思考	21
5.1 最终实验结论	21
5.2 与实验要求的对应	21
5.3 对并行化难点的理解	22
A Git 项目链接	23
B 实验结果文件组织	23

C AI 辅助编程对话记录	23
C.1 关于 MPI 实验总体规划的讨论	23
C.2 关于 qsub 脚本和测试流程的讨论	24
C.3 关于主从式任务池设计的讨论	25
C.4 关于 changed iteration 与 sweep_cap 的讨论	25
C.5 关于 local B + rotation logs 的讨论	26
C.6 关于正确性 bug 与奇异值顺序的讨论	26
C.7 关于 master_work 的讨论	27
C.8 关于 MPI+OpenMP 混合并行的讨论	27
C.9 关于 blocking 与 nonblocking 的讨论	28
C.10 关于实验结果如何解释的讨论	28
C.11 关于是否需要多 seed block 不均衡统计的讨论	29
C.12 关于报告组织的讨论	29
C.13 小结	30
D 参考文献	31

1 实验背景

1.1 SVD 分解与本实验选题

奇异值分解 (Singular Value Decomposition, SVD) 是数值线性代数中的基本矩阵分解方法。对于矩阵

$$A \in \mathbb{R}^{m \times n}, \quad m \geq n,$$

其 SVD 可写为

$$A = U \Sigma V^T,$$

其中 $U \in \mathbb{R}^{m \times m}$ 和 $V \in \mathbb{R}^{n \times n}$ 为正交矩阵, $\Sigma \in \mathbb{R}^{m \times n}$ 为非负对角矩阵, 主对角线上的元素即矩阵 A 的奇异值。SVD 广泛应用于最小二乘、降维、主成分分析、图像压缩、低秩近似和科学计算等场景 [1, 2]。

本次并行程序设计实验选择的算子为矩阵 SVD 分解。前序 SIMD 实验和 Pthread/OpenMP 多线程实验已经完成了 GKH 迭代过程中的局部循环展开、连续内存访问、线程级并行等优化。本次 MPI 实验在此基础上进一步探索多进程并行, 重点研究如何将 SVD 迭代过程中的活动子问题划分为任务, 并通过 MPI 主从式结构和任务池进行分发。

1.2 本实验采用的 SVD 实现方式

本项目中的 SVD 实现分为两个主要阶段:

1. 使用 Householder 变换将原矩阵 A 化为上二对角矩阵 B , 并同时累积左右正交变换, 使得

$$A = U B V^T.$$

2. 在上二对角矩阵 B 上执行 Golub-Kahan bidiagonal SVD 迭代, 通过 Givens 旋转不断消去超对角元素, 最终使 B 收敛为非负降序的对角矩阵。

由于上二对角化阶段当前仍主要由主进程完成, 本实验的 MPI 并行化重点放在第二阶段, 即 GKH 迭代。GKH 迭代过程中, 若某个超对角线元素足够接近 0, 则上二对角矩阵可以被分割为若干互不影响的活动块。每个非平凡活动块 $[l, r]$ 可以看作一个局部 SVD 子问题, 适合作为 MPI 任务进行分发。

1.3 MPI 编程实验目标

MPI (Message Passing Interface) 是一种面向分布式内存并行程序设计的消息传递接口标准, 常用于集群、多节点和多进程并行计算 [3, 4]。与 Pthread/OpenMP 等共享内存并行方法不同, MPI 中不同进程通常拥有独立地址空间, 需要显式使用消息通信传递数据。

本实验的主要目标如下:

1. 在 SVD GKH 迭代中实现 MPI 主从式结构;
2. 设计任务池, 将非平凡活动块作为任务动态分配给 worker;
3. 按实验手册要求改变原有迭代方法, 使 worker 在局部块中连续迭代, 直到收敛、分裂或达到 `sweep_cap`;

4. 使用 profiling 计时分析 MPI 通信、任务分发、worker 计算、master 本地计算与合并回放开销；
5. 合并前序 SIMD/内存访问优化，比较 SIMD-only 与 MPI+SIMD 的性能差异；
6. 探索不同问题规模、不同进程数、不同节点布局 and 不同线程数下的性能表现；
7. 比较 blocking 与 nonblocking 两类 MPI 通信方法；
8. 探索 MPI 与 OpenMP、编译优化等多层次优化方式的结合。

2 MPI 算法设计与实现

2.1 整体设计思路

本次 MPI 实验不是简单地把串行 SVD 程序外层套上一层多进程循环，而是围绕 GKH 迭代过程中自然产生的活动子问题进行任务级并行设计。原始 SVD 程序主要包含两个阶段：首先使用 Householder 变换将矩阵 A 化为上二对角矩阵 B ，然后在 B 上执行 Golub-Kahan bidiagonal SVD 迭代，使其逐步收敛为非负降序的对角矩阵。

在本实验中，上二对角化阶段仍由 master 进程完成，而 MPI 并行化重点放在第二阶段，即 GKH 迭代阶段。这样设计的原因有三点：

1. GKH 迭代阶段是前序实验 profiling 中的重要耗时部分，尤其在大规模矩阵上占据主要时间；
2. GKH 迭代中超对角线元素收敛到 0 后会自然产生多个相互独立的活动块，这些活动块可以作为 MPI 任务；
3. 前序 SIMD 和 OpenMP 实验已经对局部 Givens 旋转、矩阵行列更新等底层循环进行了优化，本次实验可以在这些局部优化基础上继续增加任务级并行。

因此，本实验最终形成了如图 1 所示的多层次结构：master 负责全局控制和任务池维护，worker 负责局部活动块迭代；局部块内部继续使用前序 SIMD/内存访问优化；master 端 rotation logs 回放部分还可以结合 OpenMP 行级并行。

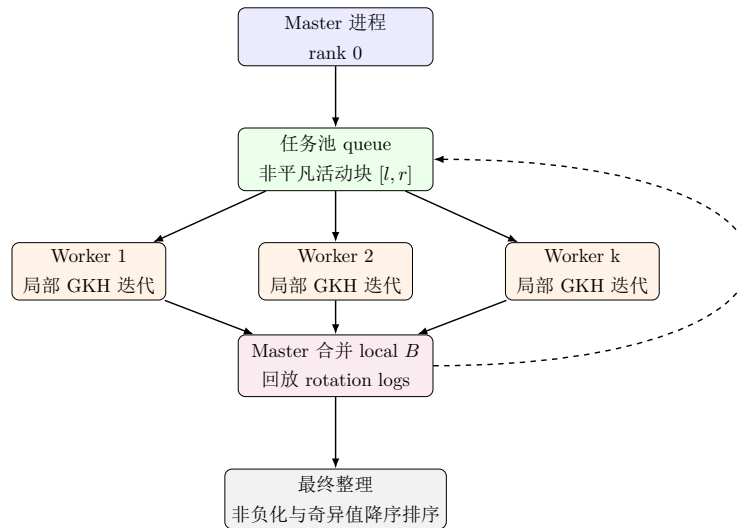


图 1: MPI 主从式任务池总体结构

2.2 项目文件结构与实现划分

本次 MPI 实验在原有 SVD 工程基础上进行扩展。为了避免 MPI 代码和串行数值计算代码混杂在一起，本实验将局部 GKH 数值过程与 MPI 调度过程分开实现。主要文件职责如表 1 所示。

表 1: MPI 实验相关文件结构

文件	作用
<code>main.cpp</code>	命令行参数解析、不同实现路径选择、正确性检查和性能输出。通过 <code>--impl</code> 参数选择 <code>serial_simd</code> 、 <code>mpi_blocking</code> 、 <code>mpi_pool</code> 、 <code>mpi_nonblocking</code> 等路径。
<code>matrix.h</code>	矩阵数据结构、矩阵乘法、随机矩阵生成和基本访问接口。后续为连续内存优化调整了部分访问方式。
<code>bidiagonalization.cpp/.h</code>	Householder 上二对角化过程，生成初始 $A = UBV^T$ 。
<code>gkh.cpp/.h</code>	串行/SIMD GKH 迭代、活动块分割、Givens 旋转、局部 block 提取与合并、rotation logs 记录与回放。
<code>gkh_mpi.cpp/.h</code>	MPI blocking、任务池、changed iteration、nonblocking、 <code>master_work</code> 等 MPI 调度实现。
<code>mpi_protocol.h</code>	MPI 任务头、结果头和消息 tag 定义，统一 master 与 worker 之间的通信协议。
<code>build_mpi.sh</code>	使用 <code>mpic++</code> 编译 MPI 版本，并支持 <code>-O2/-O3</code> 等编译选项。
<code>qsub_mpi.sh</code>	服务器 PBS/qsub 任务提交脚本，用于控制节点数、进程数、线程数和实验参数。
<code>results_lab4/</code>	本次 MPI 实验各阶段结果记录，包括 blocking、任务池、 <code>sweep_cap</code> 、 <code>np scaling</code> 、 <code>node layout</code> 、 <code>nonblocking</code> 、MPI+OpenMP 等实验。

这种划分有两个优点。第一，串行/SIMD 版本和 MPI 版本可以共享同一套局部 GKH 计算逻辑，有利于保证数值结果一致。第二，MPI 层主要负责任务管理和通信，数值计算层主要负责局部矩阵变换，便于逐步调试和定位性能瓶颈。

2.3 GKH 迭代中的活动块划分

GKH 迭代的对象是上二对角矩阵 B 。在迭代过程中，若某个超对角线元素 $B_{k,k+1}$ 足够小，则可以认为矩阵在该位置发生解耦，原问题可分为左右两个相互独立的子问题。本实验中使用如下判断判断超对角元素是否可视为 0：

$$|B_{k,k+1}| \leq \text{tol} \cdot (|B_{k,k}| + |B_{k+1,k+1}| + 1).$$

若满足该条件，则将 $B_{k,k+1}$ 置零。随后扫描整个超对角线，将连续非零超对角线连接起来的区间视为活动块。一个活动块用闭区间

$$[l, r]$$

表示，其中 l 和 r 为该子问题在全局矩阵中的列索引。当 $l = r$ 时，该块为 1×1 ，视为已收敛；当 $l < r$ 时，该块为非平凡子问题，需要继续执行 GKH 迭代。

本实验中使用 `GKHBlock` 表示活动块：

Listing 1: 活动块数据结构

```
1 struct GKHBlock
```

```

2  {
3      int l;
4      int r;
5  };

```

活动块划分是 MPI 任务池的基础。master 维护一个由非平凡 GKHBLOCK 组成的队列，worker 每次从队列中获取一个活动块并进行局部迭代。

2.4 主从式结构与任务池设计

本实验采用主从式 MPI 结构。令 rank 0 为 master，其余进程为 worker。master 的主要职责包括：

1. 执行 Householder 上二对角化，得到初始 U, B, V ；
2. 扫描 B 的超对角线，将当前问题分割为活动块；
3. 维护任务池，将非平凡活动块加入队列；
4. 向 worker 分发任务，并接收 worker 返回的局部结果；
5. 合并 local B ，回放 rotation logs 更新全局 U, V ；
6. 判断是否收敛，并在结束时整理奇异值顺序。

worker 的主要职责包括：

1. 等待 master 发送活动块任务；
2. 接收该活动块对应的 local B ；
3. 在局部块内执行若干步 GKH 迭代；
4. 记录本地迭代中产生的 Givens rotation logs；
5. 返回 local B 、rotation logs 以及可能产生的新子块信息；
6. 收到停止信号后退出。

主从式任务池的基本流程如下：

Listing 2: 主从式任务池伪代码

```

1  Master:
2      B <- bidiagonalize(A)
3      queue <- split_active_blocks(B)
4
5      while queue is not empty or some workers are active:
6          while queue is not empty and some worker is idle:
7              task <- pop(queue)
8              send task to idle worker
9
10         receive one completed result
11         merge local B
12         replay rotation logs on U and V

```

```

13     push new nontrivial child blocks into queue
14
15     send STOP to all workers
16     finalize U, B, V
17
18 Worker:
19     while true:
20         receive task header
21         if task is STOP:
22             break
23
24         receive local B
25         run local GKH iterations
26         send result header, child blocks, rotation logs and local B

```

该设计相比静态任务分配更灵活，因为活动块的数量和大小会随着迭代动态变化。任务池可以使较早完成任务的 worker 继续领取新任务，缓解不同活动块计算量不均的问题。

2.5 改变迭代方法：changed iteration

原始串行 GKH 实现通常采用全局迭代方式：每一轮扫描所有活动块，并对每个非平凡活动块执行一步 bulge chasing。这种方式在单进程中比较自然，但直接搬到 MPI 中会导致任务粒度过细。若 worker 每次只执行一步局部迭代，就需要频繁接收任务、返回结果、由 master 合并，再进入下一轮。这样通信和调度开销会非常明显。

因此，本实验按照实验手册要求改变迭代方法。worker 接收到一个活动块后，不再只执行一步，而是在局部块内连续迭代，直到满足以下条件之一：

1. 该局部块完全收敛，即所有超对角元素均可视为 0；
2. 该局部块再次分裂为多个非平凡子块；
3. 达到预设的最大局部迭代次数 `sweep_cap`。

该策略在本报告中称为 `changed iteration`。其伪代码如下：

Listing 3: `changed iteration` 局部迭代逻辑

```

1 run_local_task(B_sub, l, r, sweep_cap):
2     logs <- empty
3     local_sweeps <- 0
4
5     while local_sweeps < sweep_cap:
6         cleanup_bidiagonal(B_sub)
7         handle_diagonal_zeros_record(B_sub, logs)
8
9         blocks <- split_active_blocks(B_sub)
10
11         if all blocks are 1x1:
12             return converged result
13
14         if B_sub splits into multiple nontrivial blocks:

```



```

15         return child blocks and current local B
16
17         one_block_step_record(B_sub, logs)
18         local_sweeps <- local_sweeps + 1
19
20     return local B and original block as unfinished task

```

这里的 `sweep_cap` 是重要参数。若 `sweep_cap` 太小，则 worker 仍然频繁返回，通信次数较多；若 `sweep_cap` 太大，则单个任务可能过重，导致任务池响应变慢或负载不均。本实验通过 `sweep_cap` 调优发现，`sweep_cap=8` 在小规模和大规模测试中都比较稳定，因此最终主线采用该值。

2.6 局部 GKH 迭代与 Givens 旋转记录

在一个活动块 $[l, r]$ 内，GKH 迭代的核心是 bulge chasing。每次迭代大致分为以下步骤：

1. 根据活动块右下角 2×2 子问题计算 Wilkinson shift；
2. 对块首部构造右 Givens 旋转，引入 bulge；
3. 交替使用左 Givens 旋转和右 Givens 旋转，将 bulge 从块首追赶到块尾；
4. 在更新局部 B 的同时，维护全局关系 $A = UBV^T$ 。

在串行版本中，执行一次右旋转时会直接更新 B 和 V ；执行一次左旋转时会直接更新 B 和 U 。但在 MPI 版本中，worker 不持有完整的全局 U, V ，如果仍然传输 $U(:, l:r)$ 和 $V(:, l:r)$ ，通信量较大。因此本实验将局部迭代改为“记录版”：worker 只更新 local B ，同时记录本次局部迭代中所有 Givens 旋转，最后将 rotation logs 返回给 master。

一个 rotation log 记录如下信息：

表 2: RotationLog 记录内容

字段	说明
side	表示旋转作用于 V 还是 U 。右旋转作用于 V ，左旋转对应累积到 U 。
k0, k1	旋转作用的两列或两行编号。该编号使用全局矩阵下标。
c, s	Givens 旋转参数，用于 master 端回放更新。

这样，worker 的计算结果由三部分组成：

1. 更新后的 local B ；
2. 局部迭代产生的 rotation logs；
3. 若该块发生分裂，则返回新的非平凡子块区间。

master 接收结果后，先将 local B 合并回全局矩阵，再按顺序回放 rotation logs 更新全局 U, V ，从而保持 $A = UBV^T$ 不变。

2.7 通信数据优化：从 U/B/V 子块到 local B + rotation logs

早期 MPI 版本直接向 worker 发送 U 、 B 、 V 的局部子块，并在 worker 端更新后返回。对于大小为 $bs = r - l + 1$ 的活动块，若传输 $U(:, l:r)$ 和 $V(:, l:r)$ ，单次任务的数据量大致为：

$$O(m \cdot bs) + O(bs^2) + O(n \cdot bs).$$

当 $m \approx n$ 且矩阵规模较大时， U/V 子块传输会明显增加通信压力。

优化后，worker 只接收 local B ，返回 local B 和 rotation logs。其通信量大致变为：

$$O(bs^2) + O(q),$$

其中 q 为本次局部任务产生的 rotation log 数量。相比直接传输 U/V 子块，该方法减少了大规模全局矩阵列块的远程传输。

两种方案对比如表 3 所示。

表 3: 两种 MPI 通信数据设计对比

项目	早期 U/B/V 子块方案	local B + rotation logs 方案
发送给 worker	$U(:, l:r)$ 、 $B(l:r, l:r)$ 、 $V(:, l:r)$	$B(l:r, l:r)$
worker 返回	更新后的 U 、 B 、 V 子块	更新后的 local B 、rotation logs、child blocks
通信量	$O(mbs) + O(bs^2) + O(nbs)$	$O(bs^2) + O(q)$
master 端开销	合并 U/B/V 子块	合并 local B 并回放 rotation logs
主要瓶颈	通信数据量较大	master 端 replay/merge 较重

该优化体现了一个重要的并程序序设计权衡：减少通信量通常会增加接收端的本地计算和合并开销。本实验后续的 `master_work`、内存访问优化和 MPI+OpenMP 都是围绕这一新瓶颈继续展开的。

2.8 master_work 策略

在实际测试中，GKH 迭代的任务池并不总是很深。很多时候同时存在的非平凡活动块数量有限，即使使用多个 MPI 进程，也可能出现 worker 等待任务的情况。如果 master 始终只负责调度，那么当任务池中只有少量任务时，强制将任务发送给 worker 反而可能带来额外通信开销。

因此，本实验加入 `master_work` 策略。当任务池较浅时，master 不再强制把任务远程发送给 worker，而是直接取出任务并在本地执行与 worker 相同的局部 changed iteration 逻辑。其基本思路如下：

Listing 4: master_work 策略伪代码

```

1 while queue is not empty or workers are active:
2     while queue is not empty and idle worker exists:
3         if master_work is enabled and queue size is small:
4             break
5         send task to worker
6
7     if master_work is enabled and queue is not empty:
```

```

8      task <- pop(queue)
9      master computes this task locally
10     merge local B and replay logs
11     push child blocks into queue
12     continue
13
14     receive worker result
15     merge local B and replay logs

```

该策略的作用包括：

1. 减少远程任务发送次数；
2. 降低 `dispatch_ms`；
3. 避免任务池过浅时 master 只调度不计算；
4. 提高单节点资源利用率；
5. 将主要瓶颈从通信调度转移到 master 本地计算与 merge/replay。

实验结果表明，在 $n = 1000$ 、 $np = 4$ 的配置下，启用 `master_work` 后远程任务数显著减少，整体性能明显改善。因此最终版本默认启用 `master_work=1`。

2.9 MPI blocking 与 nonblocking 通信探索

本实验实现了 blocking 和 nonblocking 两类 MPI 通信方法。

blocking 主线使用 `MPI_Send` 和 `MPI_Recv` 完成任务分发与结果接收。该方法实现简单、逻辑清晰，便于保证正确性，也是最终主线版本采用的通信方式。

nonblocking 探索版本在任务池框架基础上使用 `MPI_Irecv` 预先挂起 worker 返回结果头的接收，并通过 `MPI_Test` 或 `MPI_Waitany` 检查哪个 worker 完成。其核心目的是减少 master 对某个 worker 的阻塞等待。当前 nonblocking 版本的特点如下：

1. 非阻塞化的是结果头接收，即 `MPIResultHeader`；
2. 后续 payload，如 local B 、rotation logs 和 child blocks，仍使用 `MPI_Recv` 接收；
3. worker 端仍复用原有 changed iteration 逻辑；
4. 该版本主要作为 MPI 不同编程方法探索，而不是最终性能主线。

实验表明，nonblocking 版本能够保证正确性，但没有超过 blocking 主线。其原因是最终主线中 `master_work` 已经将远程发送任务数降到较低水平，通信等待不再是主要瓶颈。当前更重要的耗时来自 master 本地计算和 rotation logs replay。因此，nonblocking 探索说明：对于本 SVD GKH 任务，优化任务粒度、减少通信次数和降低 master 端 replay 开销，比简单替换阻塞通信为非阻塞通信更关键。

2.10 MPI 与 SIMD、OpenMP 的结合

本实验中的 MPI 版本不是从未优化串行代码重新开始，而是在前序 SIMD 和多线程实验的基础上继续扩展。这样做符合实验手册中“将 SIMD 实验代码合并到 MPI 实验中”的要求。

当前代码中的 SIMD/内存访问优化主要包括：

- 在 `apply_left_rows` 和 `apply_left_rows_range` 中保留手动循环展开，减少循环控制开销；
- 在 `apply_right_cols` 和 `apply_right_cols_range` 中使用指针和连续行访问，减少反复调用矩阵访问函数的开销；
- 对 local B 的提取与合并使用按行连续复制，提高缓存局部性；
- 通过 `-O2/-O3` 编译优化，使编译器进一步进行内联、循环优化和潜在自动向量化；
- 对 rotation logs replay 中单条 Givens rotation 的行循环使用 OpenMP 并行。

需要强调的是，SIMD、OpenMP 和 MPI 在本实验中作用于不同层次：

表 4: MPI、OpenMP 与 SIMD/内存优化的层次关系

优化方式	作用层次
SIMD/内存访问优化	优化单进程内部的局部行列更新、Givens rotation 和 local B 拷贝。
OpenMP	在 master 端对单条 rotation 的行循环进行共享内存并行，缓解 replay/merge 瓶颈。
MPI	在多进程之间分发非平凡活动块，实现任务级并行。
编译优化	通过 <code>-O2/-O3</code> 进一步优化局部热点循环。

因此，本项目的最终版本可以理解为“MPI + SIMD/内存优化 + OpenMP + 编译优化”的组合。MPI 解决的是较粗粒度的任务分发问题，SIMD/内存优化解决的是局部循环效率问题，OpenMP 则补充优化 master 端 rotation logs replay 中的行级并行。

2.11 主要数据结构与通信协议

为保证 MPI 通信过程清晰，本实验定义了任务头和结果头结构体。任务头主要用于告诉 worker 当前要处理哪个活动块；结果头用于告诉 master 该任务是否收敛、是否产生新子块以及 rotation logs 数量等信息。

典型任务头包含：

表 5: MPI 任务头信息

字段	作用
<code>l</code>	活动块左端点，全局索引
<code>r</code>	活动块右端点，全局索引

典型结果头包含：

表 6: MPI 结果头信息

字段	作用
<code>l,r</code>	返回结果对应的原活动块区间
<code>converged</code>	当前局部块是否已经完全收敛
<code>split_count</code>	局部块分裂出的非平凡子块数量
<code>sweep_count</code>	本次任务实际执行的局部迭代步数
<code>log_count</code>	本次任务产生的 rotation log 数量

通过固定消息 tag 区分任务、结果和停止信号，可以避免 master 与 worker 在多阶段通信中发生混淆。worker 收到 STOP tag 后退出循环，保证 qsub 作业可以正常结束。

2.12 Profiling 指标设计

为了分析 MPI 并行化效果，本实验在代码中加入了细粒度计时，而不仅仅记录总时间。主要 profiling 指标包括：

表 7: MPI profiling 指标设计

指标	含义
dispatch_ms	master 将任务发送给 worker 的累计耗时。该指标反映任务分发和通信开销。
worker_compute_ms	worker 端局部 GKH 计算耗时总和。
master_compute_ms	启用 master_work 后，master 本地处理任务的耗时。
merge_ms	master 合并 local B 并回放 rotation logs 的耗时。
total_ms	MPI GKH 阶段总耗时。
tasks_sent	master 远程发送给 worker 的任务数量。
tasks_done	总完成任务数量，包括 master 本地处理和 worker 返回。
queue_rounds	任务池处理轮次。
max_queue_size	任务池最大深度，用于观察同时可分发任务数量。

这些指标帮助定位了不同阶段的主要瓶颈。例如，早期版本中 dispatch_ms 和任务发送次数较高，说明通信调度开销明显；引入 local B + logs 和 master_work 后，tasks_sent 显著降低，但 master_compute_ms 和 merge_ms 成为主要耗时，说明瓶颈转移到 master 端局部计算与回放过程。

2.13 小结

本节介绍了本次 MPI 实验的核心算法设计。总体上，本实验不是简单对串行 SVD 外层循环进行并行化，而是针对 GKH 迭代中活动块动态分裂的特点，设计了主从式任务池；又通过 changed iteration 增大任务粒度，通过 local B + rotation logs 减少通信数据量，通过 master_work 缓解任务池深度不足带来的通信开销，通过 OpenMP 和 SIMD/内存访问优化继续降低 master 端局部计算和 replay 开销。

这一设计过程体现了 MPI 程序优化中的一个核心思想：并行性能不仅取决于进程数量，还取决于任务划分、通信量、同步方式、负载均衡和单进程内部计算效率。后续实验结果将进一步验证这些设计选择的性能影响。

3 实验环境与测试规范

3.1 服务器与编译方式

本实验在课程提供的 MPI 集群服务器上完成，使用 PBS/qsub 提交作业。编译器使用 mpic++，满足 MPI 实验手册要求。主线编译脚本如下：

Listing 5: MPI 编译脚本核心命令

```
1 mpic++ -O2 -std=c++17 -fopenmp -pthread
2 main.cpp gkh.cpp gkh_mpi.cpp bidiagonalization.cpp
```

```
3 -o main
```

在编译优化探索中，还比较了 `-O2` 和 `-O3`。由于 `qsub` 任务可能被调度到不同计算节点，本文没有将 `-march=native` 作为主线实验选项，以避免编译节点和运行节点 CPU 指令集不一致带来的风险。

3.2 运行参数与测试模式

程序支持命令行参数控制测试模式与实现版本，主要参数如表 8 所示。

表 8: 主要命令行参数

参数	说明
<code>--mode</code>	<code>check</code> 或 <code>bench</code> ，分别用于完整正确性检查和单规模性能测试
<code>--impl</code>	选择实现路径，如 <code>serial_simd</code> 、 <code>mpi_blocking</code> 、 <code>mpi_pool</code> 、 <code>mpi_nonblocking</code>
<code>--n</code>	<code>bench</code> 模式下随机方阵规模
<code>--seed</code>	随机矩阵种子，保证可复现性
<code>--repeat</code>	重复次数
<code>--sweep-cap</code>	<code>changed iteration</code> 中每个局部任务最多连续迭代步数
<code>--profile</code>	是否输出 MPI profiling 信息
<code>--master-work</code>	是否启用 master 参与计算策略
<code>--omp-threads</code>	OpenMP 混合并行线程数

典型 `qsub` 运行配置如下：

Listing 6: 典型 `qsub` 运行参数

```
1 NP=4
2 MODE=bench
3 IMPL=mpi_pool
4 N=1000
5 SEED=20260408
6 REPEAT=1
7 SWEEP_CAP=8
8 PROFILE=1
9 MASTER_WORK=1
10 OMP_THREADS=2
```

3.3 正确性检查指标

为验证 SVD 结果正确性，程序在 `root` 进程上检查以下指标：

1. 重构误差：

$$\frac{\|A - U\Sigma V^T\|_F}{\|A\|_F + 1};$$

2. U 的正交性误差：

$$\|U^T U - I\|_F;$$

3. V 的正交性误差:

$$\|V^T V - I\|_F;$$

4. 对角结构误差, 即非对角元素最大绝对值;

5. 奇异值是否非负;

6. 奇异值是否按降序排列;

7. GKH 迭代是否在最大迭代次数内收敛。

采用的主要阈值为:

表 9: 正确性检查阈值

指标	阈值
相对重构误差	10^{-8}
U/V 正交性误差	10^{-7}
对角结构误差	10^{-10}
降序误差	10^{-12}

3.4 Profiling 设计

本实验没有只记录总运行时间, 而是在 MPI 主线中加入了内部 profiling。主要记录项如表 10 所示。

表 10: MPI profiling 指标

指标	含义
dispatch_ms	master 分发任务给 worker 的耗时
worker_compute_ms	worker 端局部计算耗时总和
master_compute_ms	master 本地处理任务的耗时
merge_ms	master 合并 local B 和回放 rotation logs 的耗时
total_ms	GKH MPI 阶段总耗时
tasks_sent	master 远程发送任务数量
tasks_done	总完成任务数量, 包括 master 本地任务和 worker 任务
queue_rounds	任务池处理轮次
max_queue_size	任务池最大深度

这些 profiling 指标用于分析通信、计算、合并与调度之间的开销分布。

4 正确性验证与服务器性能结果

4.1 实验数据说明与分析口径

本节给出 MPI 版本的正确性验证与服务器性能测试结果。实验数据主要来自服务器 qsub 任务输出文件以及程序内部 profiling 输出。为了便于分析, 本实验将总时间分为两个主要阶段:

$$T_{\text{total}} = T_{\text{bidiag}} + T_{\text{gkh}},$$

其中 T_{bidiag} 表示 Householder 上二对角化阶段耗时, T_{gkh} 表示 Golub-Kahan bidiagonal SVD 迭代阶段耗时。由于本次 MPI 并行化主要作用于 GKH 迭代阶段, 因此本文在分析加速效果时重点关注 T_{gkh} , 同时也给出端到端总时间作为补充。

加速比定义为:

$$S_{\text{gkh}} = \frac{T_{\text{baseline,gkh}}}{T_{\text{optimized,gkh}}}, \quad S_{\text{total}} = \frac{T_{\text{baseline,total}}}{T_{\text{optimized,total}}}.$$

需要说明的是, qsub 服务器环境存在一定运行波动, 不同任务可能被调度到不同节点, 节点负载也可能不同。因此本文更关注不同优化策略带来的稳定趋势, 而不是只比较单个时间数值。对于每组实验, 本文尽量结合 profiling 指标共同分析, 例如 `dispatch_ms`、`master_compute_ms`、`merge_ms`、`tasks_sent` 和 `max_queue_size` 等。

4.2 正确性验证结果

首先对 `serial_simd` 与 MPI 路径进行正确性检查。check 模式包含固定 5×5 矩阵、随机 8×8 矩阵、近秩亏损 10×8 矩阵、随机 10×8 矩阵和随机 1000×1000 矩阵。测试矩阵既包含小规模固定矩阵, 也包含随机矩阵和近秩亏损矩阵, 可以覆盖一般随机输入、病态相关列以及大规模输入等情况。

正确性检查主要包含以下指标:

1. GKH 迭代是否在最大迭代次数内收敛;
2. 相对重构误差:

$$\frac{\|A - U\Sigma V^T\|_F}{\|A\|_F + 1};$$

3. U 和 V 的正交性误差:

$$\|U^T U - I\|_F, \quad \|V^T V - I\|_F;$$

4. Σ 是否为对角结构;
5. 奇异值是否非负;
6. 奇异值是否按降序排列。

以 $n = 1000$ 随机矩阵为例, 典型正确性结果如表 11 所示。

表 11: $n = 1000$ 典型正确性结果

指标	数值
converged	yes
relative reconstruction error	3.51609×10^{-13}
$\ U^T U - I\ _F$	2.08929×10^{-13}
$\ V^T V - I\ _F$	2.11765×10^{-13}
diagonal structure error	0
descending order error	0
nonnegative diagonal	yes
passed	1/1

可以看到, 相对重构误差和正交性误差均在 10^{-13} 量级, 远小于本文设置的阈值。这说明 MPI 并行化、local $B + \text{rotation logs}$ 、`master_work` 和 OpenMP replay 均未破坏 SVD 分解的数值正

确性。尤其是 rotation logs 方案并不直接在 worker 端更新全局 U, V ，而是在 master 端回放旋转，因此正确性验证能够说明该设计仍然保持了

$$A = UBV^T$$

这一核心不变量。

4.3 MPI 策略演化实验

本实验并不是一次性得到最终版本，而是经历了多个策略演化阶段。初始版本只验证阻塞式主从通信是否正确，随后逐步引入任务池、changed iteration、sweep_cap 调优、local $B + \text{logs}$ 、master_work、OpenMP 和 nonblocking 探索。表 12 总结了主要演化路线。

表 12: MPI 策略演化总结

阶段	代表配置	代表性能	主要结论
serial_simd	$n = 1000, np = 1$	GKH 21197.7 ms	提供 SIMD-only 基线，保留前序局部循环和内存访问优化。
mpi_blocking	$n = 64, np = 2$	GKH 47.2634 ms	能保证 MPI 正确性，但每个任务只做一步，通信和调度频繁。
mpi_pool step queue	$n = 64, np = 2$	GKH 50.6351 ms	引入任务池后调度更灵活，但任务粒度仍然过细。
changed iteration	$n = 64, cap = 4/8$	GKH 约 10 ms	局部连续迭代显著减少返回次数，是早期最关键的优化。
sweep_cap 调优	$cap = 1, 2, 4, 8$	cap=8 较稳	在通信次数、任务粒度和负载均衡之间折中。
local $B + \text{logs}$	$n = 1000$	通信量下降	减少 U/V 子块传输，但瓶颈转向 master 端 replay/merge。
master_work	$n = 1000, np = 4$	GKH 约 15–16 s	大幅减少远程任务发送，适合任务池较浅的 GKH 过程。
MPI+OpenMP	$np = 4, OMP = 2$	GKH 11709.0 ms	降低 master compute 和 merge/replay 开销。
nonblocking	MPI_Irecv 探索	未超过 blocking	通信等待不是最终主瓶颈。

从该演化过程可以看出，本实验的优化重点逐步发生变化。早期的主要问题是通信频繁，因此 changed iteration 和 sweep_cap 的收益明显；中期通过 local $B + \text{logs}$ 降低通信量后，master 端 replay/merge 成为新瓶颈；后期则通过 master_work、OpenMP 和编译优化继续缓解 master 端瓶颈。因此最终性能提升不是单一技术带来的，而是任务粒度、通信量、master 策略、内存访问和线程级并行共同作用的结果。

4.4 sweep_cap 调优结果

sweep_cap 控制每个局部任务最多连续执行多少次 GKH step。它直接决定 MPI 任务粒度：若 sweep_cap 较小，worker 会频繁返回，通信次数多；若 sweep_cap 较大，单个任务计算更充分，但可能降低任务池调度响应速度。表 13 给出了 $n = 64$ 下的调优结果。

表 13: `sweep_cap` 调优结果 ($n = 64, np = 2$)

<code>sweep_cap</code>	GKH/ms	tasks_sent	queue_rounds	max_queue_size
1	56.6521	165	165	1
2	38.2872	98	98	1
4	10.0579	64	64	1
8	10.0358	63	63	1

实验结果表明, `sweep_cap` 从 1 增大到 2 时, 任务发送数从 165 降至 98, GKH 时间从 56.6521 ms 降至 38.2872 ms; 继续增大到 4 后, 任务发送数进一步降至 64, GKH 时间降至约 10 ms。此后从 4 增大到 8 时, 任务数只从 64 降至 63, 性能几乎不再改善。

这说明 `changed iteration` 的主要收益来自减少过细粒度任务造成的通信与调度开销。当 `sweep_cap` 足够大, 使一个局部任务可以连续推进到接近分裂或收敛时, 继续增大 `sweep_cap` 的边际收益会下降。综合小规模和后续大规模实验, 本文选择 `sweep_cap=8` 作为最终主线参数。

4.5 不同问题规模下的性能

为了观察问题规模对 MPI+SIMD 收益的影响, 本文比较了 `serial_simd` 和最终 MPI+SIMD 版本在不同 n 下的性能。这里 `serial_simd` 表示只运行前序 SIMD/内存访问优化后的单进程版本, MPI+SIMD 表示在同一局部 GKH 计算基础上加入 MPI 任务池、`changed iteration`、`local B` + `logs` 和 `master_work`。

表 14: 不同问题规模下 SIMD-only 与 MPI+SIMD 对比

n	SIMD GKH/ms	MPI+SIMD GKH/ms	GKH 加速比	SIMD total/ms	MPI total/ms	total 加速比
64	4.95899	5.83220	0.85	5.72898	6.58899	0.87
256	404.956	469.375	0.86	458.066	521.554	0.88
512	6706.49	4366.27	1.54	7149.43	4814.41	1.49
1000	20912.2	15133.3	1.38	25254.29	19027.52	1.33

从表 14 可以看出, 问题规模对 MPI 是否有效影响很大。在 $n = 64$ 和 $n = 256$ 时, MPI+SIMD 反而慢于 SIMD-only。这是因为小矩阵的局部 GKH 计算量较小, MPI 进程启动、任务调度、结果回收、`local B` 合并和 `logs replay` 等额外开销无法被计算量摊薄。

当规模增大到 $n = 512$ 后, MPI+SIMD 的 GKH 阶段达到 1.54 倍加速, 端到端达到 1.49 倍加速; 在 $n = 1000$ 时, GKH 阶段达到 1.38 倍加速, 端到端达到 1.33 倍加速。这说明随着矩阵规模增加, 局部块迭代和 `replay/merge` 中的计算量变大, MPI 任务级并行开始能够覆盖通信和调度成本。

该实验也说明: MPI 并不会自动带来加速。对于本 SVD GKH 任务, 只有当问题规模足够大时, MPI+SIMD 才能优于单进程 SIMD-only; 小规模下应优先使用串行/SIMD 或多线程版本。

4.6 不同进程数下的性能

为了考察 MPI 进程数对性能的影响, 本文在 $n = 1000$ 下测试了不同 np 配置。结果如表 15 所示。

表 15: 不同进程数下性能对比 ($n = 1000$)

配置	np	GKH/ms	total/ms	说明
serial_simd	1	21197.7	25579.97	单进程基线
mpi_pool, master_work=1	2	18686.7	22408.14	小幅加速
mpi_pool, master_work=1	4	15012.7	18718.14	当前较优
mpi_pool, master_work=1	8	19627.3	23125.44	进程过多后通信/竞争增加

结果表明, MPI 加速并不随进程数线性增长。从 $np = 1$ 到 $np = 4$, GKH 时间由 21197.7 ms 降至 15012.7 ms, 说明适量 MPI 进程确实可以带来收益; 但当进程数继续增加到 $np = 8$ 时, GKH 时间反而上升到 19627.3 ms。

该现象与 GKH 迭代的任务结构有关。活动块的数量由上二对角矩阵超对角线收敛情况决定, 并不是随着 MPI 进程数增加而增加。profiling 中常见的 `max_queue_size` 较小, 说明同时可分发的非平凡任务有限。当进程数超过可用任务并行度时, 额外进程会带来更多调度、同步和资源竞争开销, 却不能获得足够计算任务。因此, 本实验中 $np = 4$ 比 $np = 8$ 更合适。

4.7 不同节点布局下的性能

在固定 $np = 4, n = 1000, \text{sweep_cap}=8, \text{master_work}=1$ 的条件下, 本文比较了不同 PBS 节点布局。结果如表 16 所示。

表 16: 不同节点布局性能对比

节点布局	GKH/ms	total/ms	master_compute/ms	merge/ms	tasks_sent
1 node $\times$ 4 ppn	15863.9	19186.21	7146.44	8026.02	14
2 nodes $\times$ 2 ppn	13035.0	16054.70	5857.84	6583.50	14
4 nodes $\times$ 1 ppn	11314.7	14662.85	3559.94	7156.59	14

本组实验中, 多节点少进程布局表现更好。由于最终版本启用了 `master_work`, 远程发送任务数只有 14, 说明跨节点通信次数并不多。在这种情况下, 多节点布局的通信劣势没有充分显现, 反而可能因为减少单节点内多个进程对 CPU、cache 和内存带宽的竞争而获得更好性能。

尤其是 4 nodes $\times$ 1 ppn 的配置中, `master_compute_ms` 从单节点的 7146.44 ms 降至 3559.94 ms, 说明节点资源竞争可能显著影响 master 本地计算速度。当然, 服务器负载和调度环境存在波动, 因此该结果不宜简单推广为所有环境下多节点一定更优。更准确的结论是: 在本实验集群和当前任务发送次数较少的最终 MPI 版本中, 多节点少进程布局表现更好。

4.8 MPI+OpenMP 混合并行

由于 local $B + \text{rotation logs}$ 优化后, master 端的 local B 合并和 rotation logs replay 成为主要瓶颈, 因此本文进一步引入 OpenMP, 对单条 Givens rotation 的行循环进行并行。需要注意的是, rotation logs 之间存在顺序依赖, 不能简单并行处理整个 log 序列; 但对于单条 rotation, 不同行的两列更新相互独立, 因此可以使用 OpenMP 并行。

在单节点 `nodes=1:ppn=8` 下, 比较 MPI-only 与 MPI+OpenMP, 如表 17 所示。

表 17: MPI+OpenMP 混合并行结果

np	OMP 线程	GKH/ms	total/ms	master_compute/ms	merge/ms	GKH 加速比
4	1	15656.3	19667.74	7957.23	6857.70	1.00
4	2	11709.0	15674.78	5638.62	5476.05	1.34
2	4	12305.5	16750.41	5563.75	6051.11	1.27

结果表明, MPI+OpenMP 能够明显改善最终版本的 master 端瓶颈。与 MPI-only 相比, $np = 4$, $OMP = 2$ 将 GKH 时间从 15656.3 ms 降至 11709.0 ms, 达到约 1.34 倍加速; 同时 `master_compute_ms` 从 7957.23 ms 降至 5638.62 ms, `merge_ms` 从 6857.70 ms 降至 5476.05 ms。

这说明当前瓶颈确实集中在 master 端局部计算和 logs replay, 而 OpenMP 能够利用单节点内多核资源加速这些行级循环。不过, $np = 2$, $OMP = 4$ 并没有超过 $np = 4$, $OMP = 2$, 说明线程数并非越多越好。过多 OpenMP 线程可能增加线程调度和内存带宽竞争, 因此需要在 MPI 进程数与 OpenMP 线程数之间折中。

4.9 blocking 与 nonblocking 通信对比

为探索不同 MPI 编程方法, 本文实现了一个 nonblocking 版本: master 使用 `MPI_Irecv` 预先挂起结果头接收, 并通过 `MPI_Test/MPI_Waitany` 检查完成状态。表 18 比较了 blocking 主线与 nonblocking 结果头接收版本。

表 18: blocking 与 nonblocking 通信对比

impl	master_work	GKH/ms	total/ms	dispatch/ms	master_compute/ms	merge/ms
<code>mpi_pool</code>	1	11718.8	15720.91	0.994	4992.19	6168.94
<code>mpi_nonblocking</code>	1	13782.2	18224.84	0.499	7541.46	5570.31
<code>mpi_pool</code>	0	17080.2	21559.17	1704.63	0	5874.49
<code>mpi_nonblocking</code>	0	17458.2	21495.57	1763.62	0	5869.34

实验结果表明, nonblocking 版本能够保证正确性, 但没有超过 blocking 主线。在 `master_work=1` 的最终配置下, nonblocking 虽然将 `dispatch_ms` 从 0.994 ms 降至 0.499 ms, 但 GKH 总时间反而从 11718.8 ms 增加到 13782.2 ms。这说明 dispatch 已经不是最终版本的主要瓶颈, 降低这一部分开销对总时间帮助有限。

在 `master_work=0` 的通信较频繁配置下, blocking 与 nonblocking 的 GKH 时间分别为 17080.2 ms 和 17458.2 ms, 二者非常接近, nonblocking 仍未取得明显优势。这是因为当前 nonblocking 实现只对结果头进行非阻塞接收, payload 数据、local B 、rotation logs 等仍需要同步接收; 同时 master 端的 merge/replay 仍然是主要开销。

因此, 本实验对 nonblocking 的结论是: 对于当前 SVD GKH 实现, 非阻塞通信可以作为 MPI 编程方法探索, 但它不是最终性能主线。相比简单替换 blocking 为 nonblocking, 减少任务数量、增大任务粒度、降低 master 端 replay/merge 开销更加关键。

4.10 编译优化与 SIMD 选项探索

为了分析底层编译优化对最终 MPI+OpenMP 版本的影响, 本文比较了 `-O2` 与 `-O3` 两种编译选项。该实验固定使用最终 MPI+OpenMP 配置, 即 $n = 1000$ 、 $np = 4$ 、`OMP_THREADS=2`、`sweep_cap=8`、

`master_work=1`。结果如表 19 所示。

表 19: 编译优化等级对比

编译选项	GKH/ms	total/ms	master_compute/ms	merge/ms	tasks_sent	GKH 加速比
-O2	16206.5	19590.38	8539.08	6890.51	14	1.00
-O3	13194.9	16963.27	6438.59	6034.76	14	1.23

从结果看，-O3 将 GKH 阶段耗时从 16206.5 ms 降至 13194.9 ms，取得约 1.23 倍加速；端到端总时间从 19590.38 ms 降至 16963.27 ms，取得约 1.15 倍加速。进一步观察 profiling 指标可以发现，-O3 并没有改变任务池结构：`tasks_sent` 仍然为 14，说明算法流程、任务数量和调度结构保持一致。因此性能提升主要来自单个任务内部执行效率提高。

具体而言，`master_compute_ms` 从 8539.08 ms 降至 6438.59 ms，下降约 24.6%；`merge_ms` 从 6890.51 ms 降至 6034.76 ms，下降约 12.4%。这说明当前最终版本中的局部 Givens rotation、local B 合并和 rotation logs replay 等热点循环仍然能从更激进的编译器优化中受益。-O3 可能带来了更充分的函数内联、循环优化、寄存器分配优化和潜在自动向量化。

该实验也说明，MPI 优化和 SIMD/编译优化并不是互相替代的关系。MPI 主要减少任务级计算时间，OpenMP 缓解 master 端行级循环瓶颈，而编译优化进一步提升局部热点循环的单核效率。因此，最终性能提升来自多层次优化叠加。

4.11 最终推荐配置与性能总结

综合上述实验，本文最终推荐的 MPI 主线配置如表 20 所示。

表 20: 最终推荐配置

项目	推荐配置
MPI 实现	<code>mpi_pool</code>
通信方式	<code>blocking MPI_Send/MPI_Recv</code>
局部迭代方式	<code>changed iteration</code>
<code>sweep_cap</code>	8
通信数据	local B + rotation logs
master 策略	<code>master_work=1</code>
单节点混合同行	<code>np = 4, OMP = 2</code>
多节点布局	4 nodes $\times$ 1 ppn 表现较好

总体来看，本实验的主要性能规律如下：

1. 小规模矩阵下 MPI 开销较大，SIMD-only 更合适；
2. 中大规模矩阵下 MPI+SIMD 可以获得明显收益；
3. 单纯增加进程数不能保证加速
4. `changed iteration` 和 `sweep_cap` 调优有效降低了任务返回次数；
5. local B + logs 降低通信量，但使 master 端 replay/merge 成为新瓶颈；
6. `master_work` 能显著减少远程任务发送，适合任务池较浅的 GKH 迭代；

7. OpenMP 和 -O3 能进一步缓解 master 端热点循环瓶颈；
8. nonblocking 通信正确但没有超过 blocking 主线，说明最终瓶颈已经不在简单通信等待上。

因此，本实验最终得到的并行化思路不是“堆更多进程”，而是通过任务粒度控制、通信量压缩、master 本地计算、线程级并行和编译优化共同改善 SVD GKH 迭代性能。

5 结论与思考

5.1 最终实验结论

本次 MPI 编程实验围绕矩阵 SVD 分解中的 GKH 迭代阶段进行了主从式 MPI 并行化。实验完成了任务池设计、changed iteration、sweep_cap 调优、local B + rotation logs 通信优化、master_work、MPI+OpenMP、blocking/nonblocking 通信对比以及编译优化探索。

主要结论如下：

1. **MPI 主从式任务池能够正确并行化 GKH 活动块迭代。**所有主要配置均通过 SVD 重构误差、正交性误差、对角结构和奇异值顺序检查。
2. **changed iteration 是降低通信开销的关键。**让 worker 在局部块内连续迭代，比每步返回一次更适合 MPI。
3. **local B + rotation logs 显著降低了通信数据量。**但同时将瓶颈转移到 master 端的合并和旋转回放。
4. **master_work 对本问题非常有效。**由于任务池深度有限，master 参与计算可以减少远程发送任务数量，降低 dispatch 开销。
5. **MPI 加速不是线性的。**GKH 迭代中的可并行活动块数量有限，增加进程数并不一定带来性能提升。
6. **MPI 与 OpenMP、SIMD/内存优化可以互补。**MPI 负责任务级并行，OpenMP 优化 master 端行循环，SIMD/编译优化提升局部热点循环效率。
7. **nonblocking 通信正确但不是当前主瓶颈。**在最终版本中，通信等待已经不是主要耗时来源，master 端计算和 merge/replay 更值得优化。

5.2 与实验要求的对应

本实验对课程要求的覆盖情况如表 21 所示。

表 21: 实验要求完成情况

要求	完成情况	说明
实现主从式 MPI 结构	已完成	rank 0 为 master，其余为 worker
实现任务池	已完成	使用 queue 动态维护非平凡活动块
改变迭代方法	已完成	局部连续迭代直到收敛、分裂或达到 sweep_cap

要求	完成情况	说明
profiling	已完成	记录 dispatch、worker、master、merge、tasks 等指标
合并 SIMD 实验代码	已完成	保留前序 GKH 局部循环和内存访问优化
比较 SIMD 与 MPI+SIMD	已完成	在不同规模下比较 serial_simd 与 mpi_pool
不同问题规模	已完成	n=64/256/512/1000
不同进程数	已完成	np=1/2/4/8
不同节点布局	已完成	1 node、2 nodes、4 nodes 对比
不同线程数	已完成	MPI+OpenMP 对比 OMP=1/2/4
MPI 不同策略	已完成	blocking、task pool、changed iteration、nonblocking
复杂性与性能分析	已完成	分析任务粒度、通信量、merge/replay 和 master 瓶颈

5.3 对并行化难点的理解

本实验最重要的体会是：并行程序的性能不只取决于“是否使用了 MPI”，更取决于任务粒度、通信量、负载均衡和算法结构是否适合并行。

SVD GKH 迭代并不是天然容易线性扩展的任务。虽然上二对角矩阵可以通过超对角线近零位置分割为若干活动块，但在很多迭代阶段，同时存在的非平凡活动块数量有限。因此，简单地把每个活动块分给不同进程并不能充分利用大量 MPI 进程。若每个任务只执行一步迭代，通信开销会掩盖计算收益；若任务太粗，又可能造成负载不均。

因此，本实验最终采用多层次优化策略：先通过 changed iteration 增大任务粒度，再通过 local B + logs 降低通信量，然后通过 master_work 避免任务池浅时的无效远程发送，最后用 OpenMP 和编译优化缓解 master 端 replay 瓶颈。这一过程比单纯增加进程数更有效。

A Git 项目链接

GitHub 项目仓库: <https://github.com/ZR-1N/Parallel-Course-Experiments>

实验分支: `svd-lab4-mpi`

B 实验结果文件组织

本次 MPI 实验结果主要保存在 `results_lab4` 目录中, 主要子目录如表 22 所示。

表 22: 实验结果目录组织

目录	内容
<code>01_correctness</code>	正确性验证记录 (已涵盖在其他文件夹)
<code>02_simd_mpi_compare</code>	SIMD-only 与 MPI+SIMD 对比
<code>03_mpi_blocking</code>	阻塞通信 baseline 结果
<code>04_mpi_task_pool</code>	任务池与 changed iteration 初步结果
<code>05_sweep_cap</code>	sweep_cap 调优实验
<code>06_np_scaling</code>	不同进程数扩展性实验
<code>07_node_layout</code>	不同节点布局实验
<code>08_nonblocking</code>	nonblocking 通信探索
<code>09_masterwork</code>	master work 功能探索 (已涵盖在其他文件夹)
<code>10_mpi_openmp</code>	MPI+OpenMP 混合并行实验
<code>11_size_scaling</code>	不同问题规模实验
<code>12_compile_simd</code>	编译优化与 SIMD/体系结构相关实验
<code>13_final</code>	MPI 策略演化总结

C AI 辅助编程对话记录

本实验过程中使用 AI 工具辅助完成了实验规划、代码结构分析、MPI 任务池设计、调试定位、性能数据整理和报告初稿撰写。AI 工具主要起到“讨论方案、解释现象、整理实验思路”的作用, 最终代码均由本人结合实验手册要求进行修改、编译、测试和确认。

本节记录若干具有代表性的对话片段。以下内容并非逐字完整聊天记录, 而是根据实际对话整理出的关键问答, 用于反映 AI 辅助在实验推进过程中的作用。

C.1 关于 MPI 实验总体规划的讨论

我:

这次实验是矩阵 SVD 分解的 MPI 编程实验。我们前面已经做过 SIMD、Pthread 和 OpenMP, 现在要做 MPI。主要是主从式结构、任务池、改变迭代方法、profiling, 还要和 SIMD 代码合并。我现在有点乱, 不知道应该先做什么, 怎么一步一步推进。

AI:

你现在不要一上来就追求最终加速。MPI 实验最容易乱, 因为它既涉及代码结构, 又涉及 qsub 脚本、节点分配、数据记录和报告分析。建议分阶段做:

第一步先把串行/SIMD 的正确性路径稳定下来, 确认 `serial_simd` 在 `check` 模式下能通过所有矩阵, 包括 1000×1000 。

第二步做一个最简单的 MPI blocking baseline。这个版本可以先不追求性能, 只要 rank 0 做主进程, worker 能接收任务、返回结果, 并且最终正确性通过即可。它的价值是验证 MPI 工程结构、编译脚本、qsub 脚本和通信协议都没问题。

第三步实现任务池。不要静态分配, 因为 GKH 迭代中的活动块会动态变化。master 维护一个 queue, worker 空闲时领取任务, 这样后面才能讨论任务分配方法和性能。

第四步按照手册要求改变迭代方法。不要让 worker 每次只做一步 GKH step, 而是让 worker 在局部 block 里连续迭代, 直到局部收敛、再次分裂或者达到 `sweep_cap`。这一步很重要, 因为它直接减少通信次数。

第五步做 profiling。这里 profiling 不一定必须用 VTune 或 perf, 可以在代码中记录 `dispatch_ms`、`worker_compute_ms`、`merge_ms`、`tasks_sent` 等指标。关键是报告中能说明瓶颈在哪里。

第六步再做优化, 比如 local $B + \text{rotation}$ logs、`master_work`、MPI+OpenMP、blocking/non-blocking 对比。每做一轮都要保存结果, 不要反复覆盖。

所以顺序应该是: 正确性 $\rightarrow$ blocking baseline $\rightarrow$ task pool $\rightarrow$ changed iteration $\rightarrow$ sweep_cap $\rightarrow$ 通信数据优化 $\rightarrow$ master_work $\rightarrow$ MPI+OpenMP 和 nonblocking 探索。

C.2 关于 qsub 脚本和测试流程的讨论

我:

我现在目录里有 `build_mpi.sh`、`bench_mpi.sh`、`qsub_mpi.sh`, 文件脚本组织有些混乱。脚本说明中是把程序 scp 到节点再用 `mpiexec` 跑。我现在应该怎么改? 是不是每次改完一个版本, 就用 `qsub` 跑, 然后记录数据?

AI:

是的, 你这个理解是对的。MPI 实验一定要形成一个固定流程, 否则后面数据会乱。

建议你把脚本分成两个层次:

第一, `build_mpi.sh` 只负责编译。它使用 `mpic++`, 把 `main.cpp`、`gkh.cpp`、`gkh_mpi.cpp`、`bidiagonalization.cpp` 编译成 `main`。不要让它顺便跑实验。

第二, `qsub_mpi.sh` 只负责服务器实验。它里面明确写本轮参数, 比如:

```
1 NP=4
2 MODE=bench
3 IMPL=mpi_pool
4 N=1000
5 SEED=20260408
6 SWEEP_CAP=8
7 MASTER_WORK=1
8 OMP_THREADS=2
```

每次只改这些参数, 跑完之后把 `test.o` 和 `test.e` 复制到对应的 `results_lab4` 子目录里。

如果 `qsub` 任务一直处于 R 状态但没有输出, 有可能是死锁; 如果一直 Q, 要用 `qstat -f` 看原因。比如如果显示 `Not enough of the right type of nodes are available`, 说明不是死锁, 而是节点资源不够。只有任务长时间运行且没有新输出, 才考虑 `qdel`。

另外, `test.o` 是标准输出, 里面一般有 `[bench]` 和 `summary`; `test.e` 是标准错误, 里面一般有 `profiling` 行。你要两者都保存, 因为报告里需要同时用总时间和 `profiling` 指标。

C.3 关于主从式任务池设计的讨论

我：

SVD 的 GKH 迭代应该怎么设计成 MPI 任务？是不是直接把矩阵按行或按列分给不同进程？

AI：

不建议直接按行或列切分整个矩阵。GKH 迭代的核心不是普通矩阵乘法，而是对上二对角矩阵做 bulge chasing。它的并行性主要来自活动块分裂。

你的 B 是上二对角矩阵。如果某个超对角元素 $B_{k,k+1}$ 收敛到 0，那么这个位置就可以把问题切开，左右两个子块后续可以独立迭代。所以更自然的 MPI 任务不是“第几行”，而是“活动块 $[l, r]$ ”。

master 做这几件事：

1. 清理 B 中足够小的非结构元素；
2. 根据超对角线是否为 0，把 B 分割成若干 block；
3. 把所有 $l < r$ 的非平凡 block 放进任务池；
4. worker 空闲时，从任务池取一个 block 发给它；
5. worker 返回结果后，master 合并结果，并把新分裂出的 block 重新放回任务池。

这样设计比静态分配更合理。因为 GKH 迭代中 block 的数量和大小是动态变化的，不是一开始就固定的。如果你静态分配，后面很容易有的进程任务很重，有的进程没活干。任务池虽然也不能保证线性加速，但至少是适合这种动态结构的基本方法。

C.4 关于 changed iteration 与 sweep_cap 的讨论

我：

手册说要改变现有迭代方法，我理解是不是 worker 不能只做一步？那到底应该怎么改？如果做太多步，会不会导致负载不均？

AI：

你的理解是对的。原来的串行 GKH 逻辑通常是全局扫描活动块，每个非平凡块做一步，然后进入下一轮。这在单进程里可以，但在 MPI 里会导致通信太频繁。

更合适的做法是：worker 收到一个 block 后，不是只做一步，而是在局部 block 内连续迭代。停止条件有三个：

1. 局部 block 完全收敛；
2. 局部 block 分裂成多个新的非平凡 block；
3. 达到最大局部迭代步数 `sweep_cap`。

这样就叫 changed iteration。它本质上是在增大 MPI 任务粒度。原来每一步都要返回 master，现在可以一个任务做多步再返回，通信次数会明显减少。

但是你担心的也对：`sweep_cap` 不能无限大。如果太小，通信频繁；如果太大，worker 可能拿到大 block 后干很久，其他进程没法及时分到新任务。所以需要实验调参。

你可以先测 `sweep_cap=1,2,4,8`。如果数据里看到从 1 到 4 性能明显变好，而 4 到 8 变化不大，就说明 `sweep_cap=8` 已经足够了。报告里可以写：`sweep_cap` 是任务粒度和负载均衡之间的折中参数，最终选择 8 是基于实验结果。

C.5 关于 local B + rotation logs 的讨论

我：

一开始 MPI 版本是把 U 、 B 、 V 的局部子块都发给 worker，然后 worker 更新后返回。但大矩阵时好像很慢。能不能只发 B ？但是如果 worker 不更新 U, V ，最后怎么保证 $A = UB V^T$ ？

AI：

可以只发 local B ，但要记录旋转日志。

你现在的 GKH one block step 里，每次右 Givens 旋转会更新 B 和 V ，每次左 Givens 旋转会更新 B 和 U 。如果 worker 没有完整 U, V ，它确实不能直接更新全局 U, V 。但是它可以记录“我本来应该对 U 或 V 做哪些旋转”。

所以 worker 做两件事：

1. 在 local B 上正常做 GKH 迭代；
2. 每产生一个 Givens rotation，就记录一条 log，包括作用方向、全局下标 k_0, k_1 、旋转参数 c, s 。

worker 返回 local B 和 logs。master 收到后：

1. 把 local B 合并回全局 B ；
2. 按顺序回放 logs，在全局 U, V 上做相同的旋转累积。

这样可以继续保持

$$A = UB V^T.$$

这种方法的好处是通信量从传 $U(:, l : r)$ 、 $V(:, l : r)$ 变成主要传 local B 和 logs。对于大矩阵，这能减少远程传输。但是代价是 master 要回放 logs，所以后面你会看到 `merge_ms` 和 `master_compute_ms` 变成瓶颈。这个瓶颈不是 bug，而是优化后瓶颈转移了。后面可以用 `master_work`、OpenMP replay、连续内存访问和 `-O3` 来继续优化。

C.6 关于正确性 bug 与奇异值顺序的讨论

我：

小 bench 出现了 `order_err` 不为 0，其他误差都很小，说明重构和正交性没问题，但是奇异值降序不对。这个是 MPI 算错了吗？

AI：

如果重构误差、正交性误差和对角结构都没问题，只是 `order_err` 不为 0，那通常不是 GKH 数值分解本身错了，而是最终整理阶段的问题。

SVD 本身允许奇异值以任意顺序出现，只要同步交换 U, V 的列，分解仍然正确。你的检查要求是“非负且降序”，所以最后必须调用类似 `make_nonnegative_and_sort` 或 `gkh_finalize_result` 的函数。

MPI 版本里容易出现的问题是：worker 更新了 local B ，master 合并后忘记统一做最终整理，或者 blocking/pool 某个路径调用了不同的 `finalize` 逻辑。你应该保证所有路径最后都调用同一个函数：

```
1 gkh_finalize_result(U, B, V, opts.tol);
```

这个函数应该完成三件事：

1. 清理 bidiagonal 结构外的小元素;
2. 把残余超对角元素置 0;
3. 把奇异值变为非负并按降序排序, 同时同步调整 U, V 的列。

如果修完后 `recon_rel` 仍然在 10^{-13} 左右, `order_err=0`, 那就说明 MPI 路径正确。

C.7 关于 `master_work` 的讨论

我:

我们发现 `np` 增大之后不一定变快, 甚至 `np=8` 比 `np=4` 慢。任务池最大深度也很小。那是不是说明 MPI 没什么用? 还是应该优化 `master`?

AI:

这不能简单说 MPI 没用, 而是说明这个问题的可并行任务数量有限。GKH 迭代的任务来自活动块分裂, 不是你想分多少就有多少。如果某一阶段只有一两个非平凡 block, 那么即使开 8 个进程, 也不会有 8 个独立任务。

这时候继续增加进程数会带来两个问题:

1. worker 没有足够任务, 只能等待;
2. master 仍然要维护通信、合并和调度, 额外开销增加。

所以你可以做 `master_work`。也就是当任务池很浅的时候, `master` 不把任务强行发给 worker, 而是自己处理一部分任务。这样有两个好处:

1. 减少 `tasks_sent`, 降低远程通信;
2. 避免任务池很浅时 `master` 只调度不计算。

不过 `master_work` 会让 `master_compute_ms` 上升, 这是正常的。你要看总时间是否下降。如果 `tasks_sent` 大幅减少, `dispatch_ms` 降低, 总 GKH 时间下降, 那这个策略就是有效的。

报告里可以写得更准确一点: 不是说证明了手册中特殊的“只有一个非 1×1 大矩阵”的情况, 而是 profiling 显示任务池可同时分发的非平凡任务数量有限, 因此单纯增加进程数不能线性加速, `master_work` 是更适合当前实现的策略。

C.8 关于 MPI+OpenMP 混合并行的讨论

我:

现在 local $B + \text{logs}$ 后, 通信少了, 但 `merge_ms` 和 `master_compute_ms` 很高。能不能把这个再并行? 是不是用 OpenMP?

AI:

可以, 但要注意并行粒度。

rotation logs 本身是有顺序依赖的。第 i 条旋转更新后的 U, V 会影响第 $i+1$ 条旋转的输入, 所以不能简单把所有 logs 用 OpenMP parallel for 并行。

但是对于“单条 rotation”来说, 它作用在 U 或 V 的两列上, 不同行之间的更新是独立的。比如对 U 的两列 k_0, k_1 做旋转时, 每一行 i 的计算互不依赖:

$$U'_{i,k_0}, U'_{i,k_1} = f(U_{i,k_0}, U_{i,k_1}, c, s).$$

所以可以在单条 rotation 内部对行循环并行。也就是说：

```
1 for each rotation log in order:
2 #pragma omp parallel for
3 for each row:
4     update two entries
```

这样既保持 logs 顺序，又利用 OpenMP 加速每条 rotation 的行级更新。

实验上你可以测：

- MPI-only: $np = 4, OMP = 1$;
- MPI+OpenMP: $np = 4, OMP = 2$;
- 另一种混合: $np = 2, OMP = 4$ 。

如果 $np = 4, OMP = 2$ 比 MPI-only 明显快，就说明 master 端 replay 确实适合用 OpenMP 补充优化。线程数也不是越多越好，因为线程调度和内存带宽会成为新瓶颈。

C.9 关于 blocking 与 nonblocking 的讨论

我：

实验要求里提到不同 MPI 编程方法，比如 blocking 和 nonblocking。我们已经有 blocking 的 `mpi_pool` 了，要不要做 nonblocking？会不会很麻烦？

AI：

可以做一个低风险探索版，不要直接替换稳定主线。

建议新增 `mpi_nonblocking` 分支。worker 逻辑先不动，master 这边只把“接收结果头”改成非阻塞：

1. master 给 worker 发任务；
2. 对该 worker 挂一个 `MPI_Irecv`，等待它返回 `MPIResultHeader`；
3. master 循环中用 `MPI_Test` 看有没有 worker 完成；
4. 如果没有完成，而 `master_work` 开启且 queue 里有任务，master 就自己处理一个；
5. 如果必须等待，就用 `MPI_Waitany` 等任意一个 worker 完成。

注意，这个版本只是“结果头非阻塞”，payload 比如 local B 和 logs 仍然可以用 `MPI_Recv`。这样实现简单，风险低，足够作为 nonblocking 探索。

如果结果没有更快，也不是失败。你要看 profiling。如果 `dispatch_ms` 本来就很小，而 `master_compute_ms` 和 `merge_ms` 很大，那 nonblocking 就很难提升总性能。报告里可以写：nonblocking 正确，但当前瓶颈不在通信等待，而在 master 端 replay/merge。因此最终主线仍采用 blocking。

C.10 关于实验结果如何解释的讨论

我：

有些数据看起来不太符合直觉。比如 $np=8$ 反而慢，多节点有时比单节点快，nonblocking 没有 blocking 快，O3 又比 O2 快很多。这些结果该如何合理解释？

AI:

你不能只写“快了”或“慢了”，要结合 profiling 解释。

1. **np=8 反而慢**: 说明这个 GKH 任务不是进程越多越好。活动块数量有限, `max_queue_size` 小, 额外进程没有足够任务, 反而增加调度和资源竞争。
2. **多节点有时比单节点快**: 最终版本 `tasks_sent` 很少, 所以跨节点通信次数并不高。多节点少进程可能缓解单节点 CPU/cache/内存带宽竞争, 所以表现更好。但这和服务器负载有关, 报告里要谨慎说“在本实验环境下”。
3. **nonblocking 没有更快**: nonblocking 主要减少等待 worker 返回结果头的阻塞。但最终版本里 `dispatch_ms` 很小, 主要耗时是 `master_compute_ms` 和 `merge_ms`, 所以 nonblocking 不一定有收益。
4. **O3 比 O2 快**: 任务数量没变, `tasks_sent` 一样, 说明不是算法流程变了, 而是局部循环执行效率提高了。可以解释为 -O3 对 Givens rotation、local B 合并和 logs replay 等热点循环做了更激进的内联、循环优化和潜在自动向量化。

总之, 每个实验都要从“总时间 + profiling”两方面解释。报告要体现你知道瓶颈在哪里, 而不是只堆数据表。

C.11 关于是否需要多 seed block 不均衡统计的讨论

我:

手册里提到, 有些同学会遇到分割出的子矩阵规模差异极大, 甚至只有一个非 1×1 大矩阵, 其余都是 1×1 。我们之前说任务池深度浅, 这和手册说的是一回事吗? 要不要做多 seed 统计?

AI:

这两个现象有关联, 但不是完全等价。

手册说的是“分割出的子矩阵规模极不均衡”, 严格来说需要统计每轮 block size, 比如每个 seed 下每轮有多少非平凡 block, 最大 block size 是多少, 是否经常出现一个大 block 加很多 1×1 block。

你现在已有的 `max_queue_size`、`tasks_sent`、`queue_rounds` 能说明任务池并行度有限, 但不能严格证明“很多 seed 都出现一个非 1×1 大矩阵”的特殊情况。

所以如果你没有专门输出 block-size CSV, 就不要在报告里声称自己证明了手册那段特殊情况。更安全的写法是:

profiling 显示当前任务池最大深度较小, 同时可分发的非平凡任务数量有限, 因此单纯增加 MPI 进程数不能获得线性加速。

这样不会把“任务池深度浅”和“子矩阵规模极不均衡”混为一谈, 也不会被助教质疑统计指标不对应。既然我们的主要优化没有依赖这段特殊免责说明, 就不需要硬做这个实验。

C.12 关于报告组织的讨论

我:

实验做了很多, blocking、task pool、sweep_cap、np scaling、node layout、OpenMP、nonblocking、O2/O3, 还有 SIMD 对比。报告怎么组织才不会乱?

AI:

报告应该按“算法设计 → 实验规范 → 结果分析 → 总结思考”的顺序写，不要按你实际调试的时间顺序流水账。

可以这样组织：

1. 实验背景：介绍 SVD、GKH、MPI 实验目标；
2. MPI 算法设计与实现：主从式结构、任务池、changed iteration、local B + logs、master_work、nonblocking、MPI+OpenMP；
3. 实验环境与测试规范：编译脚本、qsub 参数、正确性指标、profiling 指标；
4. 正确性验证与性能结果：按实验主题分小节放表格；
5. 结论与思考：总结哪些优化有效、瓶颈如何转移、有哪些不足；
6. 附录：GitHub 链接、结果文件组织、AI 辅助记录、参考文献。

性能结果部分不要只贴表。每个表后面都要解释：

- 这个实验想验证什么；
- 数据中最关键的变化是什么；
- 变化说明了哪个瓶颈；
- 这个结论如何影响后续优化。

这样报告会显得像一个完整的实验研究过程，而不是简单运行了很多命令。

C.13 小结

总体来看，AI 工具在本实验中的作用主要体现在以下几个方面：

1. 帮助拆解 MPI 实验要求，明确主从式结构、任务池、changed iteration、profiling 和 SIMD 合并等核心任务；
2. 辅助设计 MPI 任务池、通信协议、sweep_cap 调优和 master_work 策略；
3. 在编译错误、qsub 输出异常、MPI 结果不正确等问题中提供定位思路；
4. 帮助解释实验数据，尤其是 np scaling、node layout、nonblocking 和 MPI+OpenMP 等结果背后的性能原因；
5. 辅助整理 README 和实验报告初稿，使实验过程和结论更清晰。

但 AI 工具没有直接替代实验过程。所有代码修改均由本人完成，所有实验数据均来自本人在服务器上编译、提交 qsub、保存输出并整理得到。AI 给出的建议也需要结合实际编译结果、正确性检查和 profiling 数据进行筛选和修正。

D 参考文献

参考文献

- [1] Gene H. Golub and Christian Reinsch. Singular value decomposition and least squares solutions. *Numerische Mathematik*, 14(5):403–420, 1970.
- [2] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.
- [3] William Gropp, Ewing Lusk, and Anthony Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 2 edition, 1999.
- [4] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard Version 4.1*, 2023.